

Replication :

Def : keeping a copy of the same data on multiple machines that are connected via a network

↓ why

1. keep data geographically close to users
2. increase availability (system works even if some parts fail)
3. to scale out the number of machines that can serve read queries (increase read throughput)

Leaders and Followers :

Every write to DB needs to be processed by every replica, or replicas will be inconsistent

↓ common solution

leader-based replication :

1. One replica is the **leader**, clients send writes here
2. Other replicas are **followers**, leader sends the data change to all its followers as part of a **replication log / change stream**. Each follower takes the log and updates its local copy of the DB accordingly, by **applying writes in the same order as they were processed on the leader**.
3. When a client wants to read from DB, can query leader or follower

Synchronous vs Asynchronous Replication:

synchronous - leader waits for follower to confirm it has received the write before reporting success to user and before making write visible to other clients

- replication is usually fast, most followers apply changes in < 1 second, but there are times when a follower might be behind the leader by several mins or more

- follower is guaranteed to have an up-to-date copy of the data

- if a sync. follower doesn't respond, the write cannot be processed and the leader must block all writes until the follower is available again

not practical for all followers to be sync. replicas

Semi-synchronous:

- one follower is sync., others are async.

- if sync. follower becomes slow/unavailable, an async follower is made sync.

- guarantees two nodes with up to date data

asynchronous - leader sends message, and does not wait for a response

often, leader-based replication is configured to be completely async.

disadvantage: writes are not guaranteed to be durable

advantage: leader can continue processing writes, even if all followers are behind

Handling Node Outages :

Goal: Keep the system as a whole running despite individual node failures,
and minimize the impact of node outages

Follower failure: catch-up recovery

- follower keeps a log of data changes it has received
- from log, knows the last transaction processed before fault occurred,
thus can connect to leader and request all data changes since that last transaction

Leader failure: Failover

Process:

1. One follower needs to be promoted to leader
2. clients need to be configured to send their writes to a new leader
3. other followers need to consume data changes from new leader

Automatic failover:

1. **Determining that leader has failed** - most systems simply use a timeout
2. **Choosing a new leader** - best candidate is node with most up-to-date data
3. **Reconfiguring the system to use the new leader** - ensure that old leader becomes a follower

Things that can go wrong:

- async replica that becomes new leader may not have received all writes from leader before failure; discard old leader's unreplicated writes
- two nodes believe they are the leader (**split brain**). Data likely to be lost or corrupted

Problems with Replication Lag:

- if an application reads from an async follower, you may see outdated information if it has fallen behind

eventual consistency - if you stop writes and wait a while, the followers will eventually be consistent with the leader

replication lag - delay between write happening on the leader and being reflected on a follower

↓ Three examples when lag is so large it introduces real problems

Example 1)

- User submits a write to leader, if user views this write shortly after submitting, the new data may have not yet reached the replica they are reading from.

we need read-after-write consistency: guarantee that if user reloads the page they will always see any updates they submitted themselves

↓
no promise about other users

implementations:

- when reading something user may have modified, read from leader; otherwise read from follower → must know whether something might have been modified, without checking → ex. in social media, profile can only be edited by user ∴ always read users own profile from the leader and other peoples profiles from follower
- use other criteria to decide whether to read from leader; ex. track time of last update and for one minute after last update make all reads from leader

Example 2)

- anomaly that occurs when user reads from a fresh replica, then from a stale replica; **time appears to go backward**

we need **monotonic reads**: guarantee that users will not read older data after having previously read newer data

implementation:

- ensure each user makes their reads from the same replica (different users can read from different replicas)

Example 3)

- violation of causality; if some partitions are replicated slower than others, an observer may see a sequence of writes in the wrong order
→ **a particular problem in partitioned (sharded) databases**

We need **consistent prefix reads**: guarantee that if a sequence of writes happens in a certain order, then anyone reading those writes will see them appear in the same order

Multi-Leader Replication :

Allow more than one node to accept writes, aka **multi-leader replication** ;

In this setup each leader simultaneously acts like a follower to other leaders

Some Use Cases :

1) Multi-datacenter operation

If you have a dB with replicas in different datacenters, with single-leader config, your leader can be in only one of those datacenters

In multi-leader config, you can have a leader in each datacenter.

Within each datacenter, regular leader-follower replication is used, but between datacenters each leader replicates its changes to other leaders

Performance

Single :

- every write must go to the datacenter with leader, may add latency

Multi :

- writes can go to closest datacenter and are replicated asynchronously to other datacenters ; perceived performance may be better

Tolerance of datacenter outages

Single:

- if datacenter with leader fails, failover promotes follower in another datacenter

Multi:

- datacenters can operate independently, so replication catches up when the failed datacenter comes back online

Tolerance of network problems:

Single:

- sensitive to problems in inter-datacenter link because writes made synchronously over this link

Multi:

- temporary network interruption does not prevent writes being processed

2) Clients with offline operation

Application needs to work even when disconnected from the internet

Ex. Calendar app

- need to be able to see meetings (reads), add new meetings (writes) regardless of internet

- changes made offline need to sync to a server and be reflected on your other devices when you get connection again
- every device has own local dB that acts like a leader, and there is an asynchronous multi-leader replication process between replicas of your calendar on all devices

3) Collaborative Editing

Handling Write Conflicts:

Biggest issue with multi-leader replication is that write conflicts can occur, meaning conflict resolution is required



Both writes will be accepted and the conflict will be detected asynchronously at some later point in time

simplest strategy for dealing with conflicts is to avoid them: if application can ensure all writes for a specific record go through the same leader.

conflict won't happen → recommended approach bc conflict resolution is hard

Ex. Requests for a particular user always routed to same datacenter, and use the leader in that datacenter for reading and writing

But: if datacenter fails or user moves to a different location,
you might want to change the designated leader for a user



conflict avoidance breaks down; need to deal with possibility of
concurrent writes

In multi-leader conf., there is no defined ordering of writes, if
every replica applied writes in the order seen, the dB will end up
inconsistent

dB must resolve conflicts in a **convergent way**, meaning all replicas
must arrive at the same final value

ways of doing so:

1) Give each write a unique ID (timestamp, UUID, etc), pick
write with highest ID as winner, and discard other writes.

If timestamp is used, known as **Last-Write-Wins (LWW)**



popular but dangerously prone to data loss

2) merge values together ex. order them alphabetically, then concatenate

3) record conflict in a datastructure and write application code

that resolves the conflict later

Custom conflict resolution logic:

Since conflict resolution may depend on the application, most multi-leader replication tools let you write your own logic that can be executed on read or write:

On-read:

- detected conflicts are stored
- next time data is read, these values are returned to application and can prompt user to fix it or automatically resolve it

On-write:

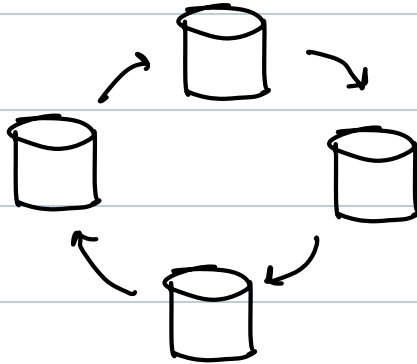
- call conflict handler as soon as conflict is detected

↓
usually a fast running
background process

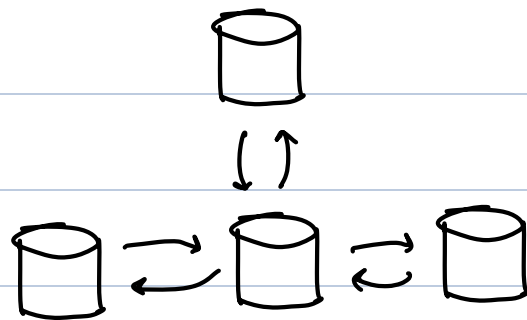
Multi-Leader Replication Topologies:

replication topology describes the communication paths which writes are propagated from one node to another

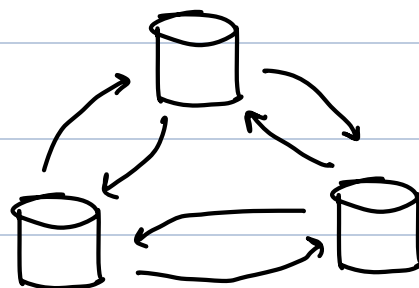
- a) **circular** - each node receives writes from only one node and forwards those writes (and its own) to one other node



- b) **star** - one designated node forwards writes to all other nodes



- c) **all-to-all** - most general topology where every leader sends writes to every other leader.



Problem with star and circular is that one node failure can disrupt the flow of replication messages to other nodes.

Problem with all-to-all is that due to varying speed of network links, some replication messages may overtake others resulting in leaders getting writes out of order.

Leaderless Replication :

no concept of leader, rather any replica can accept writes

aka **Dynamo-style** database

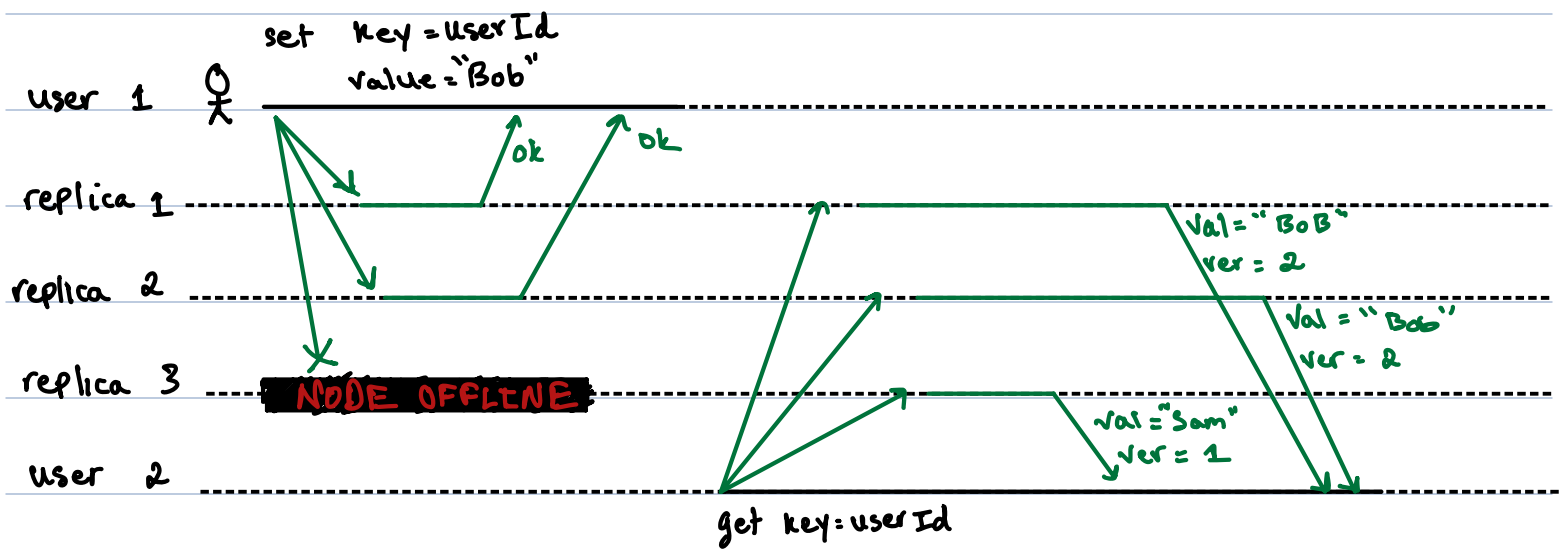
↪ idea was mostly forgotten till
Amazon brought it back for
DynamoDB

Writing to the Database When a Node is Down:

In single/multi-leader configs, if the leader goes down, failover will have to occur to accept writes again

In leaderless, failover does not exist.

Here's what happens :



client sends writes to all three replicas, and two available replicas accept, but the unavailable replica misses it.

↳ lets say 2/3 is sufficient to consider the write successful

Now the unavailable node comes back; any writes that happened while it was down are missing ∴ reading from it might give you stale data

To solve this reads are sent to all replicas, most recent version is sent back

Read repair and anti-entropy:

Two mechanisms to ensure that nodes can catch up on writes

Read repair

- when a client makes a read and sees that a node has a stale value, it writes the newer value to that replica

↓
good for frequently read data

Anti-entropy Process

- Background process that looks for differences in data and copies missing data from one replica to another

Quorums for reading and writing:

If there are n replicas, every write must be confirmed by w nodes to be considered successful and we must query r nodes for each read: As long as $w+r > n$, expect to get up-to-date value when reading

Commonly n is odd, and set $w=r=(n/2)+1$ (rounded up).

You can vary w and r for your usecase, for example setting $w=n$ and $r=1$ makes reads faster, but one unavailable replica will block all writes

Edge Cases where stale values are returned (even with $r+w > n$):

- if sloppy quorum is used, the w writes may end up on dif. nodes than the r reads
- if two writes occur concurrently
- if a write happens concurrently with a read
- if a node carrying a new value fails and its data is restored

from a replica carrying an old value

Sloppy Quorums and Hinted Handoff :

Nodes that a client cannot connect to due to network problems may prevent the client's requests from reaching quorum.

In a large cluster (w/ significantly more than n nodes) it's likely a client will be able to connect to some database nodes, just not the ones that it needs to reach quorum for a particular value.

In this case DB designers face a trade-off:

1. return errors for all requests for which we cannot reach quorum

2. accept writes and write them to some nodes that are reachable but aren't among the n nodes on which the value usually lives

aka
↓

sloppy quorum - writes and reads require w and r successful responses, but those may include nodes that are not among the designated "home" nodes for a value

hinted handoff - any writes that one node that temporarily accepted on behalf of another node are sent to the appropriate "home nodes" when those "home" nodes are reachable again

Sloppy quorums are useful for increasing write availability, however, more likely to read stale data

Detecting Concurrent Writes:

Dynamo-style DBs allow several clients to concurrently write to the same key which means conflicts will occur even if strict quorums are used.

Last-Write-Wins:

LWW is a poor choice for conflict resolution if losing writes is not acceptable

The "happens-before" relationship and concurrency:

- An operation A happens before another operation B if B knows about A, or depends on A, or builds upon A.

Two operations are **concurrent** if neither happened before the other (neither knows about the other)

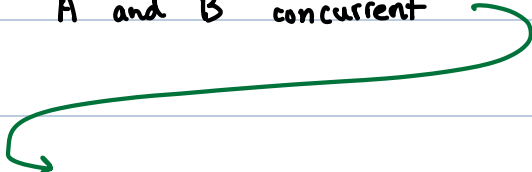
Thus when you have operations A and B

there are only three options:

1. A before B

2. B before A

3. A and B concurrent



We need an algorithm that can detect this



Algorithm:

- server maintains a version number for every key and increments the version number everytime that key is written, and stores new version number along with value written
- when a client reads a key, server returns all values that have not been overwritten, as well as the latest version number. A client MUST read a key before writing
- when client writes a key, must include version number from prior read and must merge together all values that it recieved in prior read
- when server recieves a write with a particular version number, it can overwrite all values with that version number or below, but it must keep all values with a higher version number